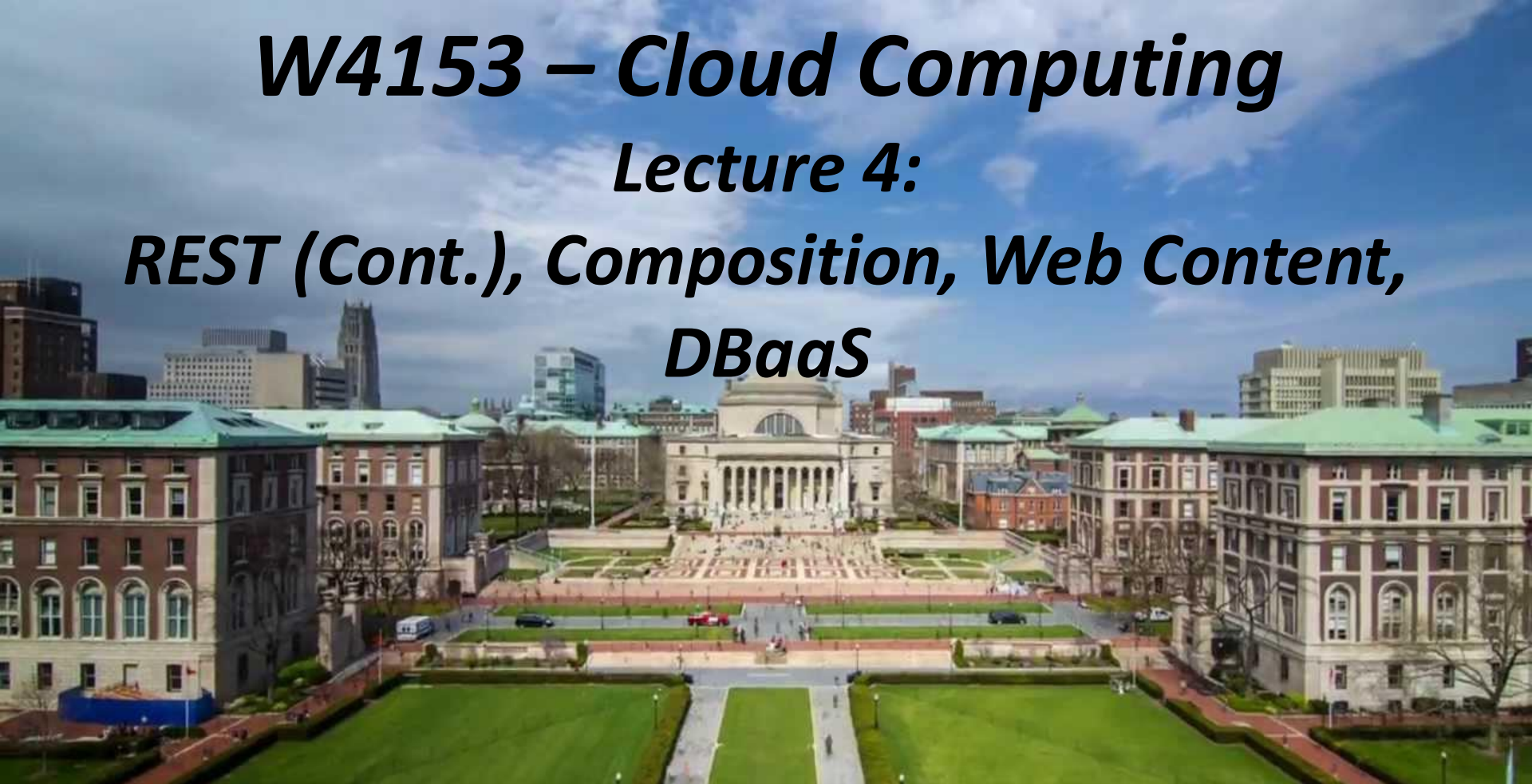




W4153 – Cloud Computing

Lecture 4:

REST (Cont.), Composition, Web Content, DBaaS

Contents

Contents

- Course updates
- Container-as-a-Service (reminder, review)
- REST continued
 - Resources, URLs and parameters.
 - Querying/filtering, pagination,
 - Linked data
- Service composition
 - Concepts
 - Orchestration
 - Choreography

Course Updates

Course Updates

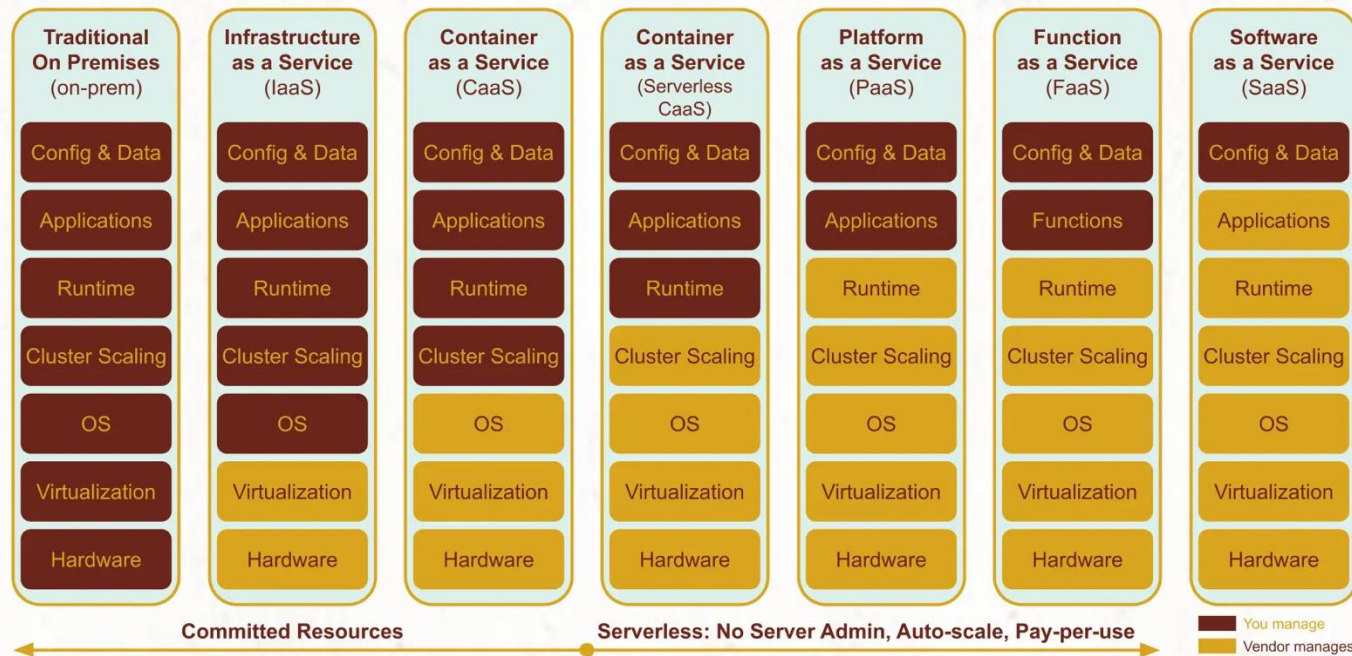
- I am finally over Covid and will be more active in engaging again.
- We will publish submission instructions for the first team sprint early this weekend. The submission format will be:
 - A presentation describing what you accomplished.
 - A document, with links, providing more details on what you accomplished.
- We will cover some concepts required for the next sprint in this lecture and the next. You can get started after today.

CaaS

Cloud Layers

Cloud Deployment Spectrum

ml4devs.com/serverless 



 © Satish Chandra Gupta, Creative Commons BY-NC-ND 4.0 International License.

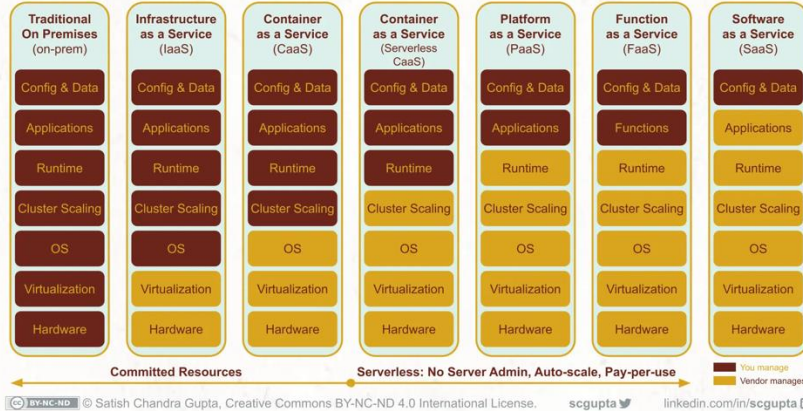
scgupta 

linkedin.com/in/scgupta 

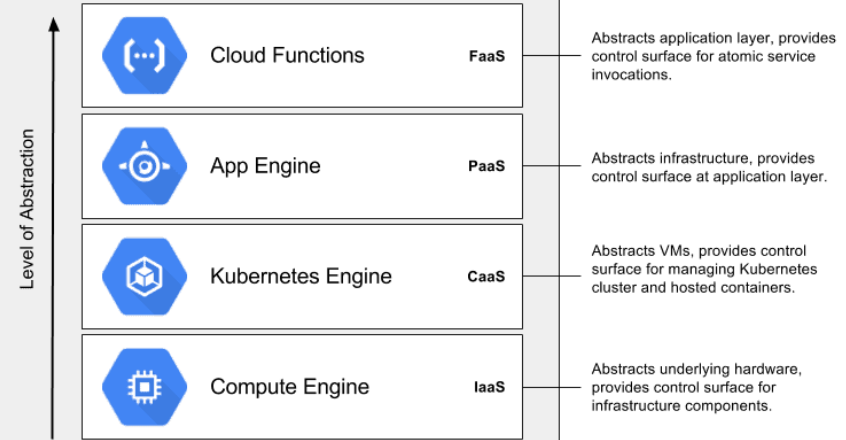
Cloud Layers

Cloud Deployment Spectrum

ml4devs.com/serverless



Google Cloud Compute Services

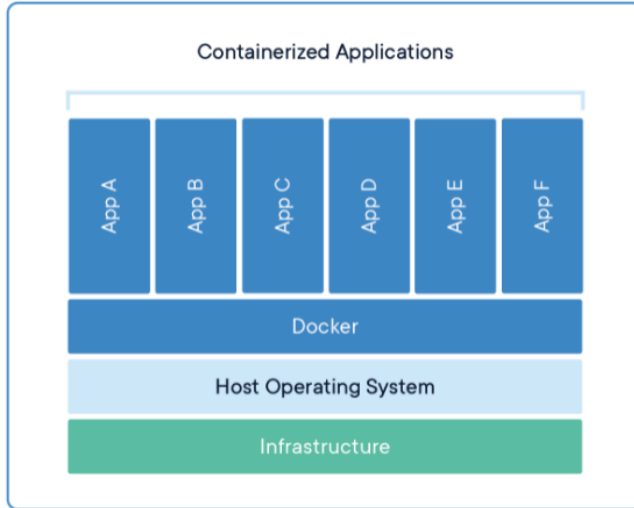


- To understand the concepts, we will “work our way up the stack.”
- This will help us understand the complexity, pros and cons.
- This means we start with *Google Cloud Platform Compute Engine*. – The basic building blocks is a “virtual machine.”

Fundamental, Recurring Problem

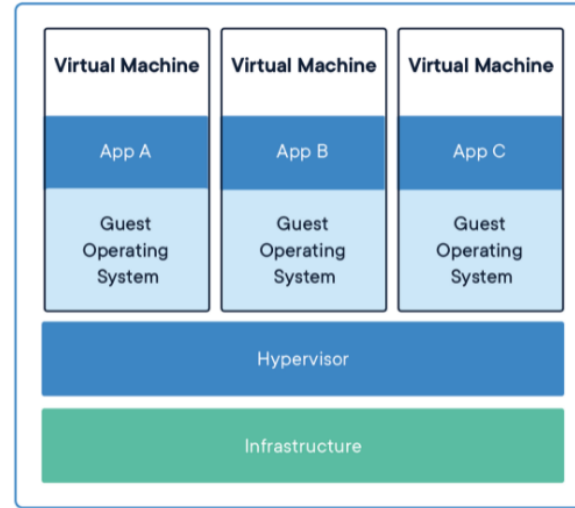
- I have one big computer and I want to run multiple applications.
- This creates several challenges, e.g.
 - Resource sharing and allocation (CPU, memory, disk,)
 - Isolation: Application A should not be able to corrupt/mess with application B's files, memory,
 - Configuration: Applications require libraries, versions of libraries, prerequisites, compiler/interpreter levels,
- Basically, I want a way to someone package or fence all of this.
We see some of the issues with Java Classpath, Python environments, ...
- There are a few ways to do this:
 - OS process and paths.
 - VMs
 - Containers

Containers and VMs



CONTAINERS

Containers are an abstraction at the app layer that packages code and dependencies together. Multiple containers can run on the same machine and share the OS kernel with other containers, each running as isolated processes in user space. Containers take up less space than VMs (container images are typically tens of MBs in size), can handle more applications and require fewer VMs and Operating systems.



VIRTUAL MACHINES

Virtual machines (VMs) are an abstraction of physical hardware turning one server into many servers. The hypervisor allows multiple VMs to run on a single machine. Each VM includes a full copy of an operating system, the application, necessary binaries and libraries - taking up tens of GBs. VMs can also be slow to boot.

Concept (from Wikipedia)

- “cgroups (abbreviated from control groups) is a Linux kernel feature that limits, accounts for, and isolates the resource usage (CPU, memory, disk I/O, network, etc.) of a collection of processes.”
- “Namespaces are a feature of the Linux kernel that partitions kernel resources such that one set of processes sees one set of resources while another set of processes sees a different set of resources.

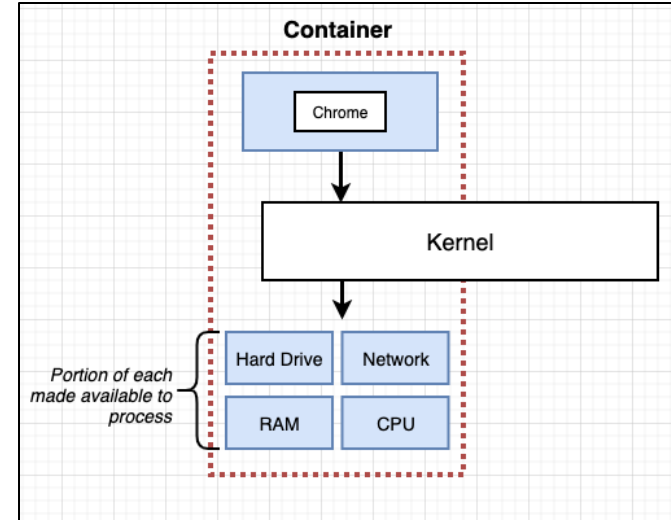
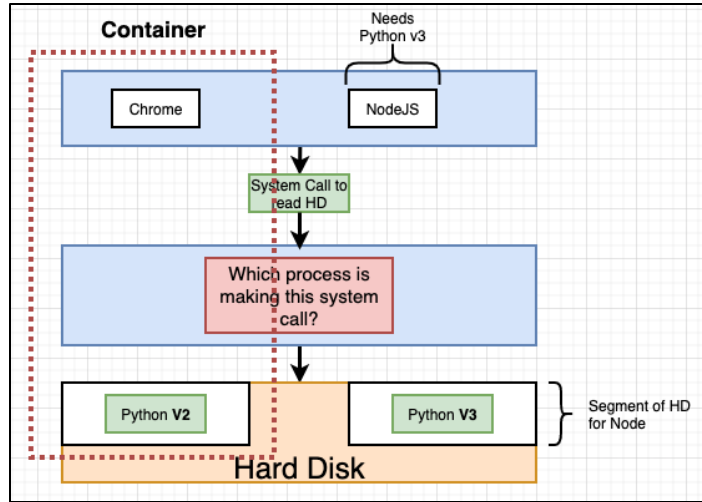
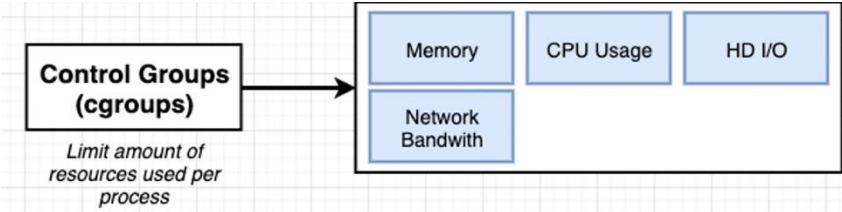
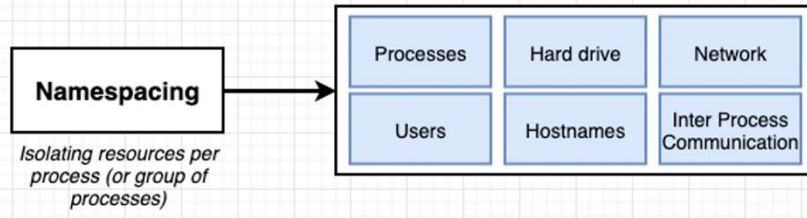
... ..

those namespaces refer to distinct resources. Resources may exist in multiple spaces. Examples of such resources are process IDs, hostnames, user IDs, file names, and some names associated with network access, ...”

Containers vs VMs

	Process	Container	VM
Definition	A representation of a running program.	Isolated group of processes managed by a shared kernel.	A full OS that shares host hardware via a hypervisor.
Use case	Abstraction to store state about a running process.	Creates isolated environments to run many apps.	Creates isolated environments to run many apps.
Type of OS	Same OS and distro as host,	Same kernel, but different distribution.	Multiple independent operating systems.
OS isolation	Memory space and user privileges.	Namespaces and cgroups.	Full OS isolation.
Size	Whatever user's application uses.	Images measured in MB + user's application.	Images measured in GB + user's application.
Lifecycle	Created by forking, can be long or short lived, more often short.	Runs directly on kernel with no boot process, often is short lived.	Has a boot process and is typically long lived.

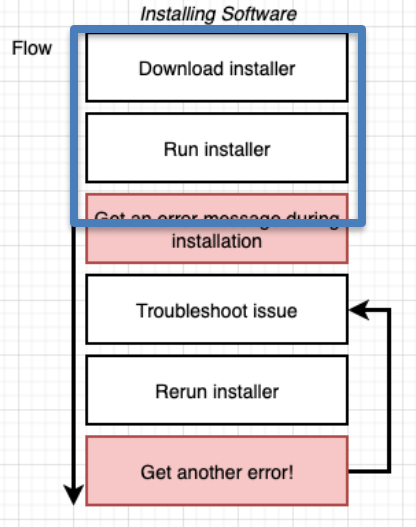
Some Container/Docker Implementation Concepts



Isolates libraries, paths, packages, PIDs,

Partitions and caps resource consumption.

Containers and Some Docker Overview



This is what Docker is trying to fix

Why use Docker?

Docker makes it really easy to install and run software without worrying about setup or dependencies

- We are familiar with the problem
 - We have a system with an OS and installed SW.
 - We need to install something new, and that requires installing prerequisites, changing versions, etc.
 - After many tries, we get the application installed only to find out we broke some other application.
- Docker and containers allow you to:
 - Develop an application or SW system.
 - Package up the entire environment (paths, versions, libs, ...) into an image that works and is self-contained.
 - Someone installing/starting your application simply gets the image from a repository and starts the image to create a running container.

How to Run Containers on GCP

- Cloud Run (serverless): Easiest. Auto-scales to zero, HTTPS by default, request-based billing. Great for web APIs (FastAPI), jobs, and event handlers.
- GKE (Google Kubernetes Engine): Full Kubernetes. Best when you need multiple services, custom networking, DaemonSets, stateful workloads, etc.
- Compute Engine (VMs + Docker): You manage the VM; simplest if you just want “a box that runs Docker.”
- Cloud Functions (2nd gen, container): Event-driven functions packaged as containers.
- ~~App Engine Flexible: Runs containers, but most new workloads should prefer Cloud Run or GKE.~~

REST Continued

REST Continued

REST (<https://restfulapi.net/>)

What is REST

- REST is acronym for **RE**presentational **State** Transfer. It is architectural style for **distributed hypermedia systems** and was first presented by Roy Fielding in 2000 in his famous [dissertation](#).
- Like any other architectural style, REST also does have it's own [6 guiding constraints](#) which must be satisfied if an interface needs to be referred as **RESTful**. These principles are listed below.

Guiding Principles of REST

- **Client-server** – By separating the user interface concerns from the data storage concerns, we improve the portability of the user interface across multiple platforms and improve scalability by simplifying the server components.
- **Stateless** – Each request from client to server must contain all of the information necessary to understand the request, and cannot take advantage of any stored context on the server. Session state is therefore kept entirely on the client.
- **Cacheable** – Cache constraints require that the data within a response to a request be implicitly or explicitly labeled as cacheable or non-cacheable. If a response is cacheable, then a client cache is given the right to reuse that response data for later, equivalent requests.
- **Uniform interface** – By applying the software engineering principle of generality to the component interface, the overall system architecture is simplified and the visibility of interactions is improved. In order to obtain a uniform interface, multiple architectural constraints are needed to guide the behavior of components. **REST is defined by four interface constraints: identification of resources; manipulation of resources through representations; self-descriptive messages; and, hypermedia as the engine of application state.**
- **Layered system** – The layered system style allows an architecture to be composed of hierarchical layers by constraining component behavior such that each component cannot “see” beyond the immediate layer with which they are interacting.
- **Code on demand (optional)** – REST allows client functionality to be extended by downloading and executing code in the form of applets or scripts. This simplifies clients by reducing the number of features required to be pre-implemented.

Resource Oriented

- There are several good, simple tutorials:
 - <https://www.restapitutorial.com/introduction>
 - <https://restfulapi.net/>
 - <https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design>The documentation agrees on a common core, but details and application are subjective.
- Identification of resources is URLs:
 - Resource collections: <http://api.contoso.com/api/students>
 - Resource: <http://api.contoso.com/api/students/dff9>
 - Links: http://api.contoso.com/api/students/dff9/courses/COMSW4153_002_2024_001/projects
- Manipulation of resources through representations: applications, Python/Java objects, relational database rows, air conditioner management APIs, are resources.
- Self-descriptive message: HTTP verbs, MIME content types, For the purposes of your project, your bodies for GET, PUT, POST have: 1) content-type application/json, 2) Use JSON for the body data.
- Hypermedia as the engine of application state – HATEOS
Just because it sounds so cool and is awesome to day.

URLs and Parameters

- Access to a resource is through a URL.
- The operations on a *collection* resource are GET, POST.
 - GET returns a “list” of resource representations and a *links section* in the body.
 - POST take a body containing the new resource data and “inserts” into the collection.
 - A collection URL is of the form /api/persons.
- The operations on an individual resource are GET, PUT and DELETE.
 - GET returns a body with the resource representation and links section.
 - PUT takes a body setting the new value for the resource.
 - DELETE does not take or return a body.
 - An individual resource URL is of the form /api/persons/dff9, where the primary key is following the collection name.
- A method on a resource has the following “parameters.”
 - Path parameters
 - Query parameters
 - Body
 - Headers

Query Parameters

2.5. Use query component to filter URI collection

Often, you will encounter requirements where you will need a collection of resources sorted, filtered, or limited based on some specific resource attribute.

For this requirement, do not create new APIs – instead, enable sorting, filtering, and pagination capabilities in resource collection API and pass the input parameters as query parameters. e.g.

```
/device-management/managed-devices  
/device-management/managed-devices?region=USA  
/device-management/managed-devices?region=USA&brand=XYZ  
/device-management/managed-devices?region=USA&brand=XYZ&sort=installation-date
```

- The basics of query string is very common, e.g. ?region=USA&brand=XYZ
- Sort/Order by is not “standard” and APIs pick and document a convention.
- Project (choosing a subset of properties) is also not standard. APIs picks and document a convention.

Query and Path Parameters

- The concept is simple.
- Some resources are containers, which means they have resources "inside" them. This is analogous to a table in a relational database.
- Consider the path `/api/persons`.
 - Path parameters: `/api/persons/dff9` →
 - GET, PUT and DELETE logically have a where clause `WHERE UNI='dff9'` if UNI is the primary key.
 - GET `/api/persons/dff9` is logically
`SELECT * FROM persons WHERE UNI='dff9'`
 - `/api/persons/` is logically "the table."
 - GET `/api/persons` is logically `SELECT * FROM persons`
 - GET `/api/persons?last_name=Ferguson&first_name=Donald` is conceptually
`SELECT * FROM persons WHERE last_name='Ferguson' AND first_name="Donald"`
 - POST is an INSERT INTO persons. The POST "generates" the key and returns the URL to the created resource in a *location header*.

HATEOAS

HATEOAS (Hypermedia as the Engine of Application State) is a constraint of the REST application architecture. HATEOAS keeps the REST style architecture unique from most other network application architectures.

The term “hypermedia” refers to any content that contains links to other forms of media such as images, movies, and text.

REST architectural style lets us use the hypermedia links in the API response contents. It allows the client to dynamically navigate to the appropriate resources by traversing the hypermedia links.

Navigating hypermedia links is conceptually the same as browsing through web pages by clicking the relevant hyperlinks to achieve a final goal.

<https://restfulapi.net/hateoas/>

<https://grapeup.com/blog/how-to-build-hypermedia-api-with-spring-hateoas/>



In a nutshell:

- REST application is basically a graph databases.
- And this explains the emergence of GraphQL.

Pagination, Hyper Linked

- Bad things may happen when a GET returns a huge result set.
- A *page* is a window on the large result set.
- *limit* is the no. of entries in the result set.
 - Client may specify a limit.
 - Server may have a default and/or maximum limit it allows.
- There are many design patterns for implementing pagination.
- Simplest is mirroring the SQL model:
 - *limit=xxx&offset=yyy*
 - The links section is the response should contain:
 - prev
 - next
 - current
 - May contain *first* and *last*.

```
{
  "data": [
    {
      "UNI": "dff9",
      "last_name": "Ferguson",
      "first_name": "Donald",
      "links": [
        {"rel": "self", "href": "/api/students/dff9"},
        {"rel": "sections", "href": "/api/enrollments?uni=dff9"}
      ]
    },
    {
      "UNI": "ff11",
      "last_name": "Ferguson",
      "first_name": "Frodo",
      "links": [
        {"rel": "self", "href": "/api/students/ff11"},
        {"rel": "sections", "href": "/api/enrollments?uni=fb11"}
      ]
    }
  ],
  "links": [
    {"rel": "current", "href": "/api/students?last_name=Ferguson&limit=2,offset=2"},
    {"rel": "prev", "href": "/api/students?last_name=Ferguson&limit=2,offset=0"},
    {"rel": "next", "href": "/api/students?last_name=Ferguson&limit=2,offset=4"}
  ]
}
```


POST -- Creating a Resource

POST methods

If a POST method creates a new resource, it returns HTTP status code 201 (Created). The URI of the new resource is included in the Location header of the response. The response body contains a representation of the resource.

If the method does some processing but does not create a new resource, the method can return HTTP status code 200 and include the result of the operation in the response body. Alternatively, if there is no result to return, the method can return HTTP status code 204 (No Content) with no response body.

If the client puts invalid data into the request, the server should return HTTP status code 400 (Bad Request). The response body can contain additional information about the error or a link to a URI that provides more details.

- POST sends a body to “create a new resource.”
- The identity/URL of the new resource is not always obvious.
- POST returns 201 Created with a link header containing the URL to the newly created resource instance.

POST

Receiving a response indicating user creation

Let's assume there's a REST API for managing users with an endpoint at `http://example.com/users`.

In this example, we send a `POST` request with the following body to create a user:

HTTP



```
POST /users HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "firstName": "Brian",
  "lastName": "Smith",
  "email": "brian.smith@example.com"
}
```

After successful user creation, the `201 Created` response will look like this:

HTTP



```
HTTP/1.1 201 Created
Content-Type: application/json
Location: http://example.com/users/123

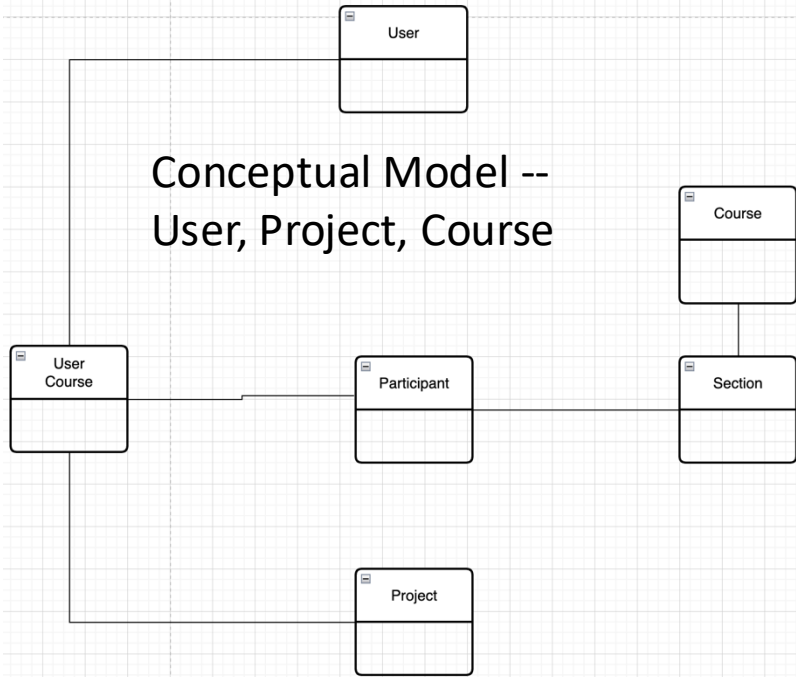
{
  "message": "New user created",
  "user": {
    "id": 123,
    "firstName": "Brian",
    "lastName": "Smith",
    "email": "brian.smith@example.com"
  }
}
```

Response

- Scroll through HTTP responses and explain ones that the projects should use.
- I expect you to return a reasonable set of responses for your methods.
You can choose the exact set but be consistent across your resource implementations.
- At a minimum, I recommend the following:
 - 200 OK
 - 201 Created
 - 202 Accepted (we will discuss this one in the next lecture)
 - 400 Bad request
 - 401 Unauthorized (next lecture)
 - 403 Forbidden (next lecture)
 - 404 Not found
 - 409 Conflict
 - 418 I'm a teapot
 - 420 Enhance your calm
 - 422 Unprocessable content
 - 501 Not implemented

Composition

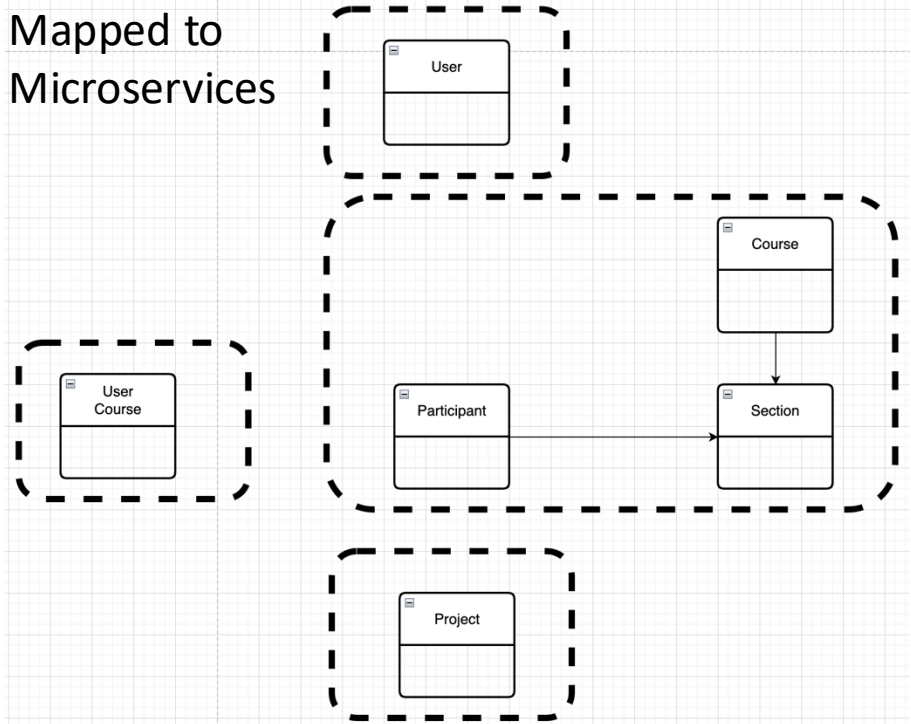
Resource Oriented and Microservices – My Project



This creates some interesting challenges, e.g.

- Joins
- Foreign Keys

Mapped to
Microservices



If the cluster of resources might be useful
in other solutions → It may be a microservice.

First Group of Microservices

- My first sprint will have 5 microservices:

- 5 core/atomic microservices:

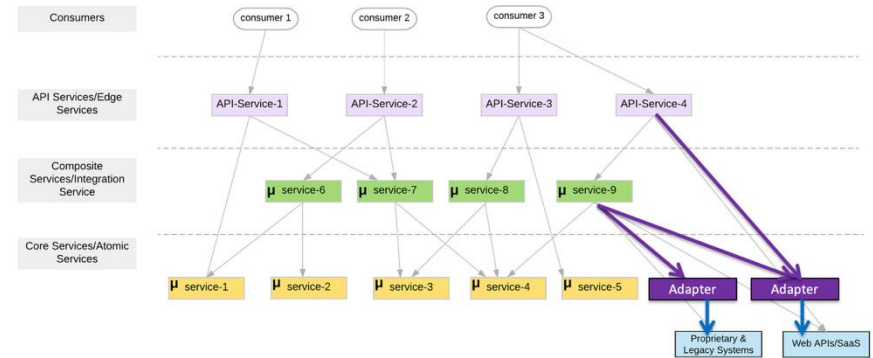
- user_info
- courseworks_adaptor
- course_info
- student_coupons
- project_teams

- 1 composite/integration microservice: core_course_management

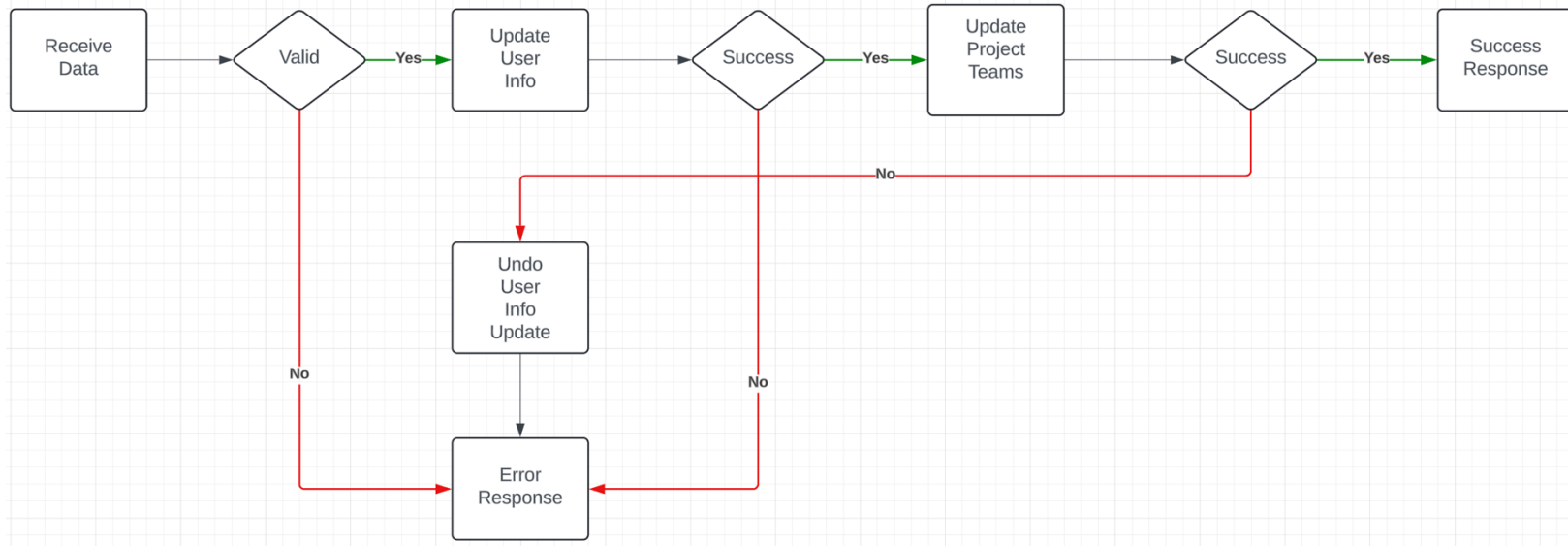
- The composite service:

- Implements “consistency” in the data in the core services, e.g. “foreign keys.”
- Eliminates the need for complex logic in the UI or client, which would be impacted by changes.
- Exposes only information from the basic services that the client requires.

- The composite service is conceptually similar to a database view.

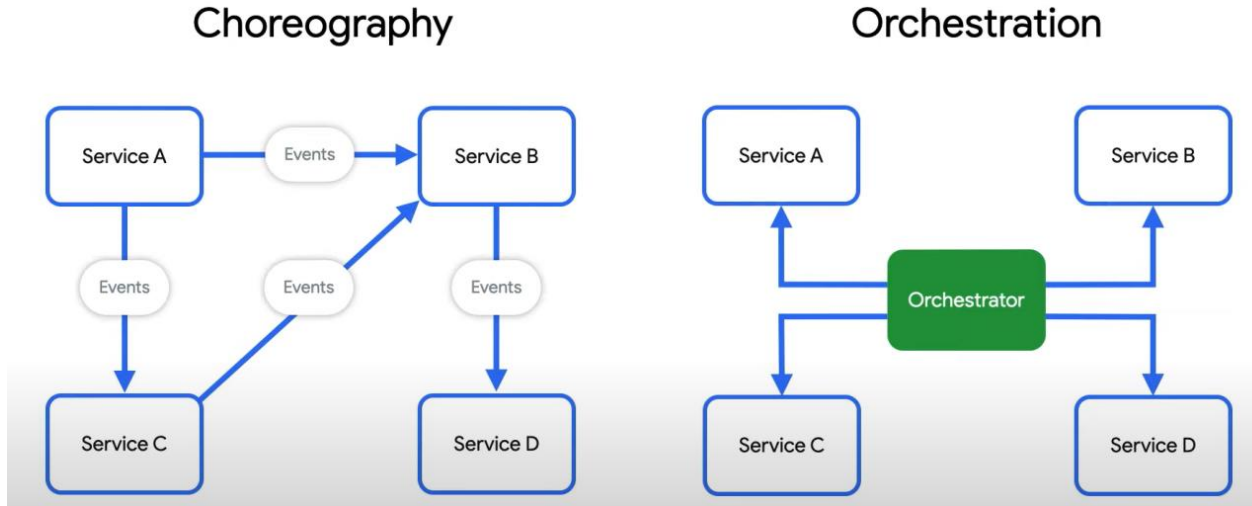


Composite Microservice Logic



- The GET on the composite invokes GET on several atomic services. This can occur synchronously or asynchronously in parallel.
- PUT, POST and DELETE are more complex. Your logic may have to undo some delegated updates that occur if others fail.
- To get started, we will just use simple logic but explore more complex approaches.

Behavioral Composition



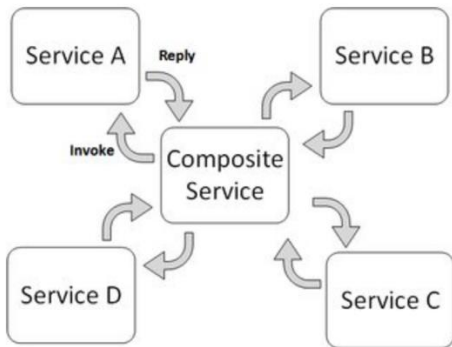
- There are two base patterns: choreography and orchestration.
- There also several different implementation technologies and choices.
- And many related concepts, e.g. Sagas, Event Driven Architectures, Pub/Sub,
- We will cover and explore incrementally, but for now we will keep it simple and do traditional “if ... then ...” code.

Concepts

Service orchestration

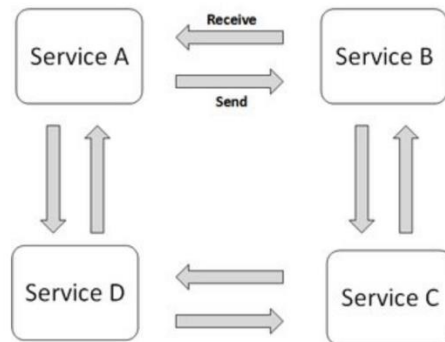
Service orchestration represents a single centralized executable business process (the orchestrator) that coordinates the interaction among different services. The orchestrator is responsible for invoking and combining the services.

The relationship between all the participating services are described by a single endpoint (i.e. the composite service). The orchestration includes the management of transactions between individual services. Orchestration employs a centralized approach for service composition.



Service Choreography

Service choreography is a global description of the participating services, which is defined by exchange of messages, rules of interaction and agreements between two or more endpoints. Choreography employs a decentralized approach for service composition.

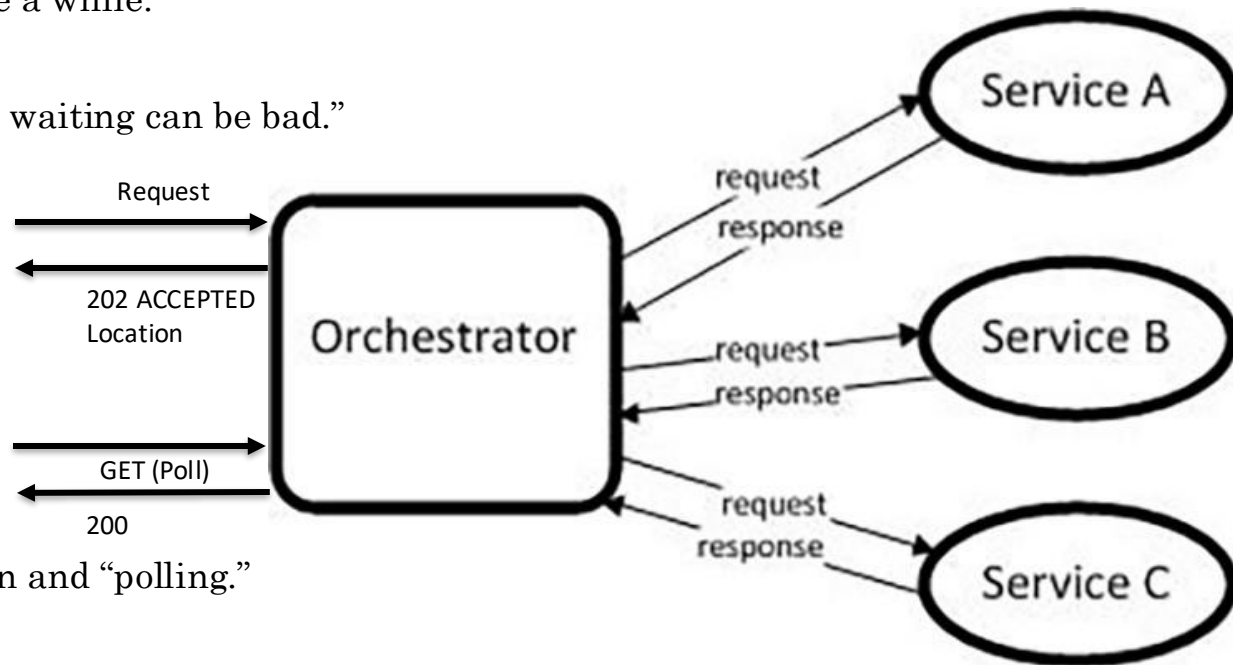


The choreography describes the interactions between multiple services, where as orchestration represents control from one party's perspective. This means that a **choreography** differs from an **orchestration** with respect to where the logic that controls the interactions between the services involved should reside.

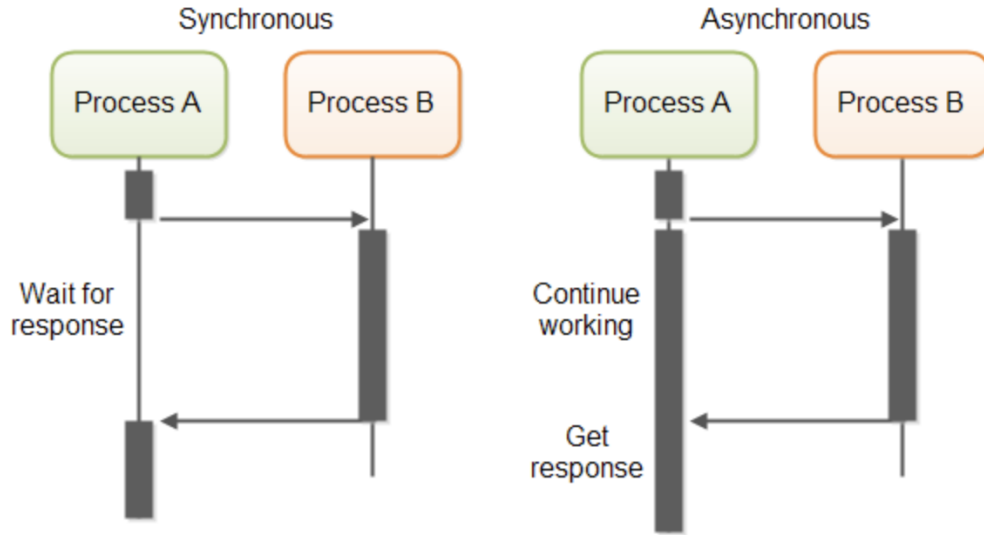
These are not formally, rigorously defined terms.

In Code: Composition and Asynchronous Method Invocation

- Composite operations can “take a while.”
- “Holding open connections and waiting can be bad.”
 - Consumes resources.
 - Connections can break.
 -
- There are two options:
 - Polling
 - Callbacks
- Our next pattern is composition and “polling.”
Create a new user requires:
 - Updating the user database.
 - Verifying a submitted address.
 - Creating an account in the catalog service.



In Code: Service Orchestration/Composition

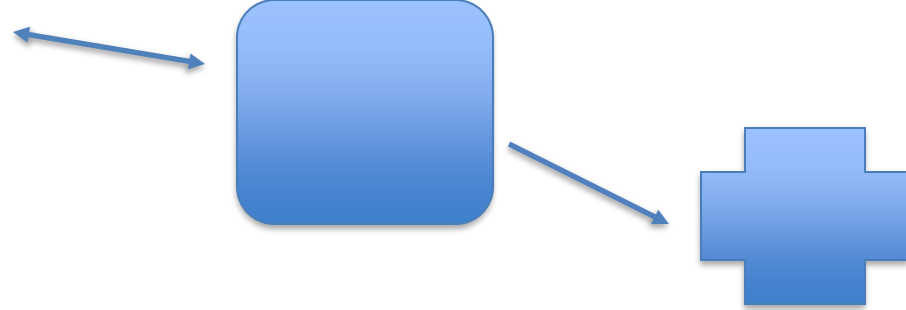
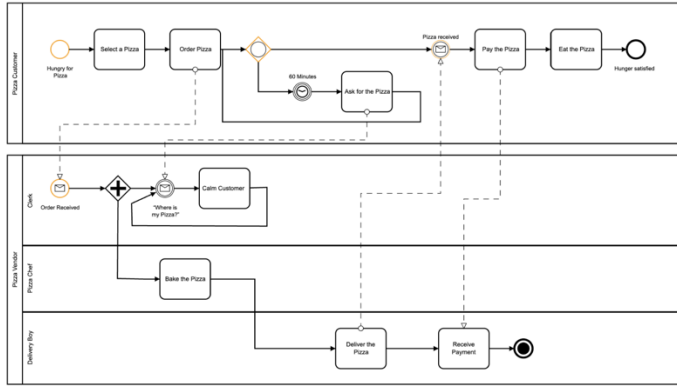


RESTful HTTP calls can be implemented in both a synchronous and asynchronous fashion at an IO level.

- One call to a composite service
 - May drive calls to multiple other services.
 - Which in turn, may drive multiple calls to other services.
- Synchronous (Wait) and calling one at a time, in order is
 - Inefficient
 - Fragile
 -
- Asynchronous has benefits but can result in complex code.

Service Orchestration

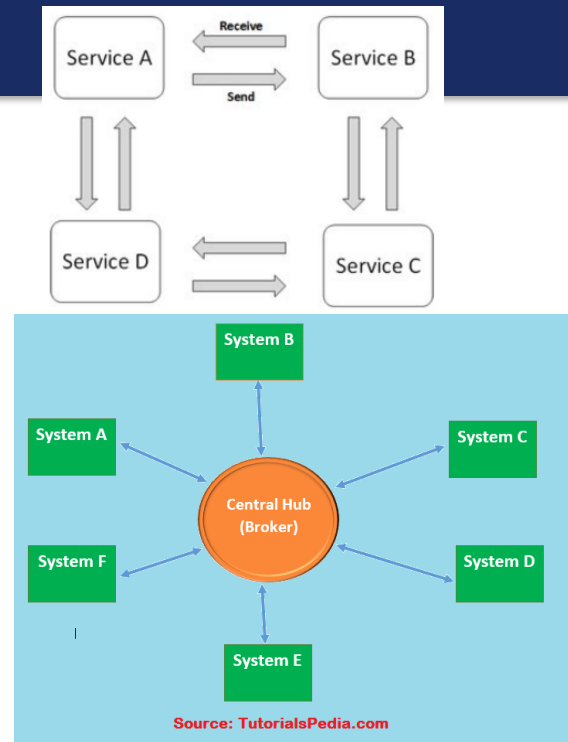
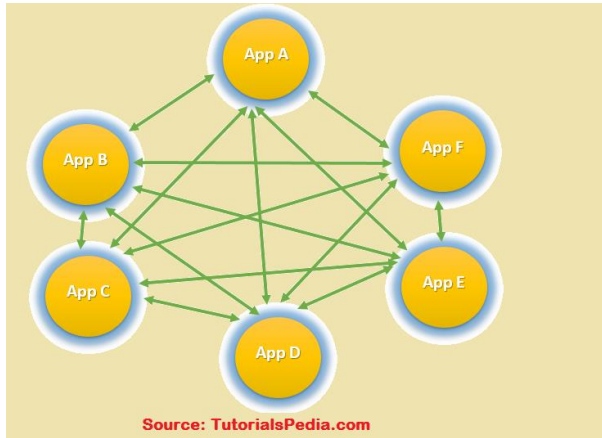
- Implementing a complex orchestration in code can become complex.
- High layer abstractions and tools have emerged, e.g. BPMN.
 - “Business Process Model and Notation (BPMN) is a graphical representation for specifying business processes in a business process model.”  [expand in new window](#)



- There are products for defining BPMN processes, generating code, executing,, e.g. Camunda: <https://camunda.com/bpmn/>
- There are other languages, tools, execution engines,

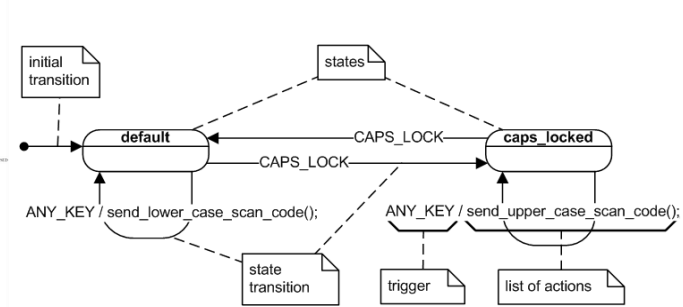
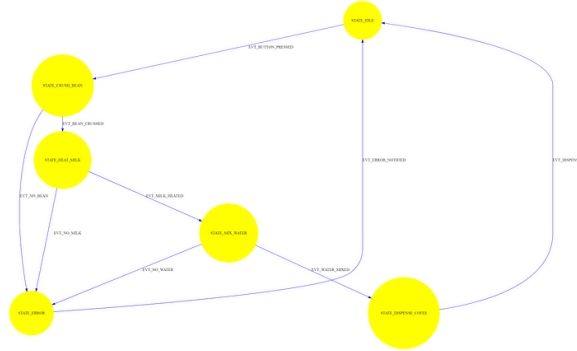
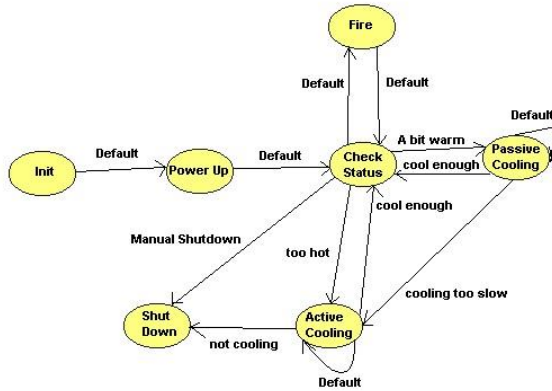
Choreography

- We saw the basic diagram, but ... this is an anti-pattern and does not scale.



- But, how do you write the event driven microservices?
 - Well, you can just write code
 - Or use a state machines abstraction.

State Machine



- A state machine is an abstraction.
- There are many models for defining state machines.
- There are also development tools, runtimes,
- We will look at a specific example: AWS Step Functions

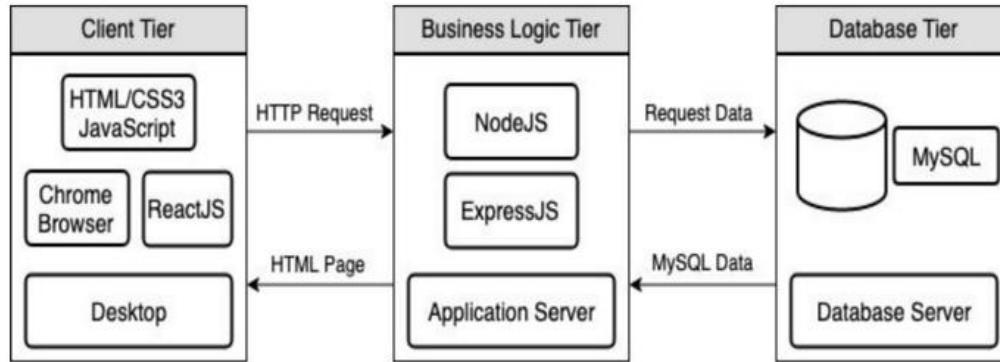
Show Code

- Show example of calling URL synchronously and asynchronously
 - `/Users/donald.ferguson/Dropbox/000/000-A-2025-Repos/sync_async/async1.py`
 - `/Users/donald.ferguson/Dropbox/000/000-A-2025-Repos/sync_async/sync1.py`
- One of the steps in your next sprint is:
 - Build a composite microservice that aggregates your basic microservices.
 - Demonstrating one method on the composite aggregating the base services synchronously.
 - Demonstrating one method on the composite aggregating the base services asynchronously.
- You are probably confused as heck right now.
 - Think of your initial microservices as base tables.
 - Think of the composite as a VIEW representing a JOIN of the base tables.
 - Your composite microservice's code implements the basic operations, and perhaps some additional logic, in the JOIN to “compose” that basic services.

Web Content (BLOB Storage)

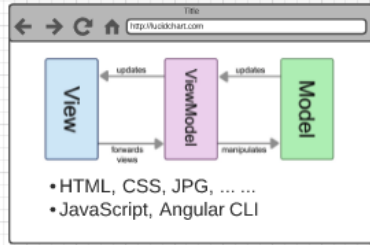
Full Stack Web Application – Reminder

<https://levelup.gitconnected.com/a-complete-guide-build-a-scalable-3-tier-architecture-with-mern-stack-es6-ca129d7df805>

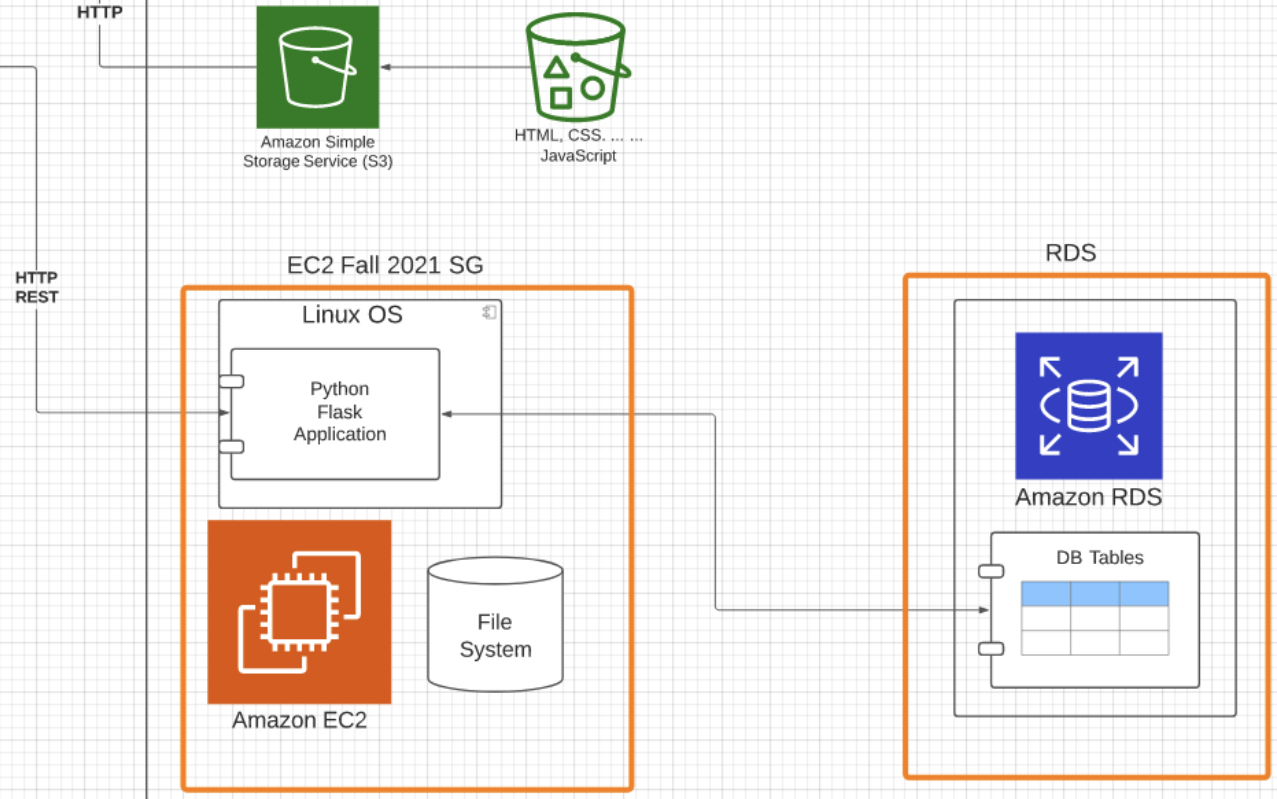


- We have seen how to use Flask (or FastAPI) to implement an endpoint (a set of paths):
 - Some of the paths can run application logic and return the result data.
 - Some can return pages that a browser can render. Basically, these paths are returning the “client application” to the browser to execute, and may have API calls in JavaScript/Typescript.
 - Do not worry about this now.
- But, running application logic and simply delivering files are two different “concerns.”
 - Having one runtime implement both would violate the SOLID principle.
 - The design points are different: “You can be a floor wax or a mouth wash, but not both.”
- So, we need a separate “web server.”

Browser



Amazon Web Services



BLOB: Binary Large Object

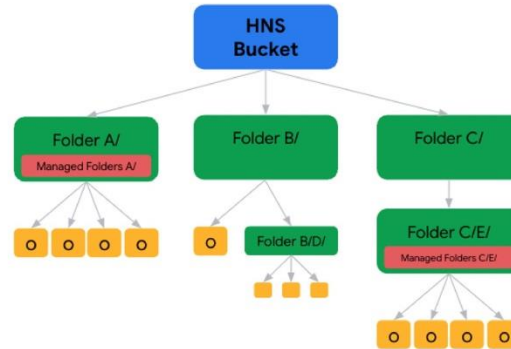
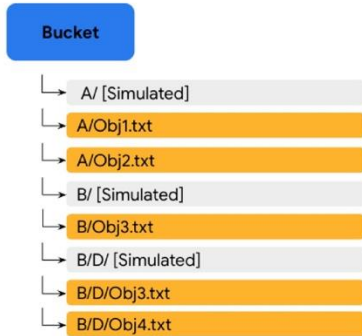
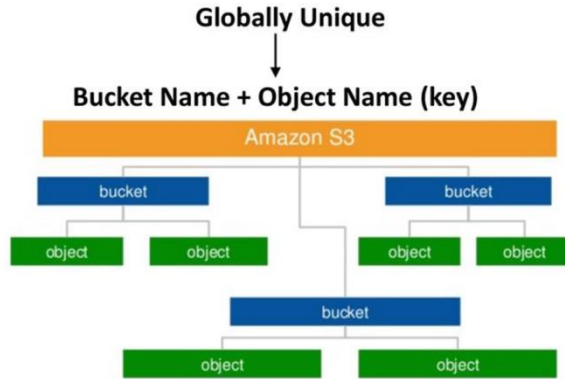
- Definitions:
 - “A binary large object (BLOB or blob) is a collection of binary data stored as a single entity. Blobs are typically images, audio or other multimedia objects, though sometimes binary executable code is stored as a blob. They can exist as persistent values inside some databases or version control system, or exist at runtime as program variables in some programming languages.”
(https://en.wikipedia.org/wiki/Binary_large_object)
 - “Blob is the data type in MySQL that helps us store the object in the binary format. It is most typically used to store the files, images, etc media files for security reasons or some other purpose in MySQL.” (<https://www.educba.com/mysql-blob/>)
- Intent:
 - File systems are not good for many application scenarios, which is why database management systems emerged.
 - Database management systems implement structured (or semi-structured) data, e.g. tables.
 - Database BLOBs were a way to have some “big things” as table or document properties.

Amazon S3



We used the equivalent capability on Azure for uploading CAD files, storing processed files and results, etc.

Concepts of S3 – Namespace



Can support a logical hierarchy with a convention on keys.

- x/y
- x/z
- x/y/z
-

Acts kind of file system like, but is really just key strings.

GCP has a very similar model.

Simple Example

- <https://e6156f20site.s3.us-east-2.amazonaws.com/index.html>

The screenshot shows the AWS S3 console for the bucket `e6156f20site`. The `Objects` tab is selected, showing a list of 13 objects. The objects are as follows:

Name	Type	Last modified	Size	Storage class
<code>assets/</code>	Folder	-	-	-
<code>favicon.ico</code>	ico	September 1, 2022, 12:45:21 (UTC-04:00)	948.0 B	Standard
<code>index.html</code>	html	September 1, 2022, 12:45:21 (UTC-04:00)	608.0 B	Standard
<code>main.js</code>	js	September 1, 2022, 12:45:22 (UTC-04:00)	43.6 KB	Standard
<code>main.js.map</code>	map	September 1, 2022, 12:45:22 (UTC-04:00)	22.3 KB	Standard
<code>polyfills.js</code>	js	September 1, 2022, 12:45:22 (UTC-04:00)	149.8 KB	Standard
<code>polyfills.js.map</code>	map	September 1, 2022, 12:45:23 (UTC-04:00)	168.2 KB	Standard
<code>runtime.js</code>	js	September 1, 2022, 12:45:23 (UTC-04:00)	6.2 KB	Standard
<code>runtime.js.map</code>	map	September 1, 2022, 12:45:23 (UTC-04:00)	6.1 KB	Standard
<code>styles.css</code>	css	September 1, 2022, 12:45:24 (UTC-04:00)	143.8 KB	Standard
<code>styles.css.map</code>	map	September 1, 2022, 12:45:24 (UTC-04:00)	761.1 KB	Standard
<code>vendor.js</code>	js	September 1, 2022, 12:45:25 (UTC-04:00)	3.1 MB	Standard
<code>vendor.js.map</code>	map	September 1, 2022, 12:45:27 (UTC-04:00)	3.2 MB	Standard

Could be hierarchical.

I need to migrate to GCP.

Static Content

- You have several options for hosting your static web content on GCP
 - Firebase Hosting – Easiest option, HTTPS + CDN, built-in SPA rewrites, great for static SPAs
 - Cloud Storage + CDN + HTTPS LB – Enterprise-scale, Google-managed SSL, integrates with load balancer infra
 - Cloud Run + NGINX – Flexible, supports custom rewrites and caching headers, serverless scaling
- There are clearly pros and cons to each approach.
- For simplicity, I suggest that you use Firebase.
- Note:
 - I followed ChatGPT's instructions for an hour and nearly had a stroke.
 - I felt like the Sorcerer's Apprentice.
 - I am going to try again over the weekend or switch to Google Cloud Store
- Note 2:
 - I started following the instructions for Google Cloud Storage
 - I nearly has another stroke.
 - I will figure this out over the weekend



DBaaS

Cloud Concepts – One Perspective

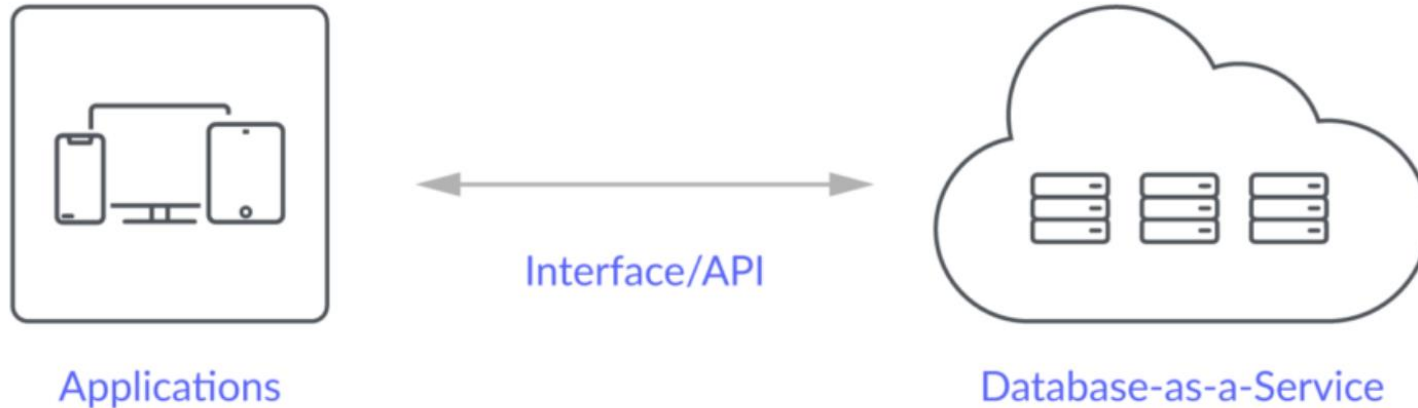
Categorizing and Comparing the Cloud Landscape

<http://www.theenterpriseearchitect.eu/blog/2013/10/12/the-cloud-landscape-described-categorized-and-compared/>

6	SaaS	Applications			End-users
5	App Services	App Services	Communication and Social Services	Data-as-a-Service	Citizen Developers
4	Model-Driven PaaS	Model-Driven aPaaS, bpmPaaS	Model-Driven iPaaS	Data Analytics, baPaaS	Rapid Developers
3	PaaS	aPaaS	iPaaS	dbPaaS	Developers / Coders
2	Foundational PaaS	Application Containers	Routing, Messaging, Orchestration	Object Storage	DevOps
1	Software-Defined Datacenter	Virtual Machines	Software-Defined Networking (SDN), NFV	Software-Defined Storage (SDS), Block Storage	Infrastructure Engineers
0	Hardware	Servers	Switches, Routers	Storage	
		Compute	Communicate	Store	

Database-as-a-Service

“A cloud database is a database that typically runs on a cloud computing platform and access to the database is provided as-a-service. There are two common deployment models: users can run databases on the cloud independently, using a virtual machine image, or they can purchase access to a database service, maintained by a cloud database provider. Of the databases available on the cloud, some are SQL-based and some use a NoSQL data model. Database services take care of scalability and high availability of the database. Database services make the underlying software-stack transparent to the user.” (Wikipedia)



Let's take a look at 3: Relational Data Service, ~~Dynamo DB~~, ~~MongoDB (Compass)~~

12 Factor Applications

12 Factor Applications

<https://dzone.com/articles/12-factor-app-principles-and-cloud-native-microser>



12 Factor App Principles

Codebase

One codebase tracked in revision control, many deploys

Dependencies

Explicitly declare and isolate the dependencies

Config

Store configurations in an environment

Backing Services

Treat backing resources as attached resources

Build, release, and, Run

Strictly separate build and run stages

Processes

Execute the app as one or more stateless processes

Port Binding

Export services via port binding

Concurrency

Scale-out via the process model

Disposability

Maximize the robustness with fast startup and graceful shutdown

Dev/prod parity

Keep development, staging, and production as similar as possible

Logs

Treat logs as event streams

Admin processes

Run admin/management tasks as one-off processes

Dependencies

II. Dependencies

Explicitly declare and isolate dependencies

Most programming languages offer a packaging system for distributing support libraries, such as CPAN for Perl or Rubygems for Ruby. Libraries installed through a packaging system can be installed system-wide (known as “site packages”) or scoped into the directory containing the app (known as “vendoring” or “bundling”).

A **twelve-factor app** never relies on implicit existence of system-wide packages. It declares all dependencies, completely and exactly, via a *dependency declaration* manifest. Furthermore, it uses a *dependency isolation* tool during execution to ensure that no implicit dependencies “leak in” from the surrounding system. The full and explicit dependency specification is applied uniformly to both production and development.

For example, Bundler for Ruby offers the `Gemfile` manifest format for dependency declaration and `bundle exec` for dependency isolation. In Python there are two separate tools for these steps – Pip is used for declaration and Virtualenv for isolation. Even C has Autoconf for dependency declaration, and static linking can provide dependency isolation. No matter what the toolchain, dependency declaration and isolation must always be used together – only one or the other is not sufficient to satisfy twelve-factor.

One benefit of explicit dependency declaration is that it simplifies setup for developers new to the app. The new developer can check out the app’s codebase onto their development machine, requiring only the language runtime and dependency manager installed as prerequisites. They will be able to set up everything needed to run the app’s code with a deterministic *build command*. For example, the build command for Ruby/Bundler is `bundle install`, while for Clojure/Leiningen it is `lein deps`.

Twelve-factor apps also do not rely on the implicit existence of any system tools. Examples include shelling out to ImageMagick or `curl`. While these tools may exist on many or even most systems, there is no guarantee that they will exist on all systems where the app may run in the future, or whether the version found on a future system will be compatible with the app. If the app needs to shell out to a system tool, that tool should be vendored into the app.

- Most application frameworks have a dependency declaration concept.
- I have been showing examples with:
 - Python import
 - requirements.txt
 - etc.
- My examples also show encapsulating libraries and APIs with Python classes.
- II. Dependencies handles “the code,” but we still need to handle the instance. There is a difference between, e.g.
 - pymysql
 - A specific connection.

III. Configurations

III. Config

Store config in the environment

An app's *config* is everything that is likely to vary between deploys (staging, production, developer environments, etc). This includes:

- Resource handles to the database, Memcached, and other backing services
- Credentials to external services such as Amazon S3 or Twitter
- Per-deploy values such as the canonical hostname for the deploy

Apps sometimes store config as constants in the code. This is a violation of twelve-factor, which requires **strict separation of config from code**. Config varies substantially across deploys, code does not.

A litmus test for whether an app has all config correctly factored out of the code is whether the codebase could be made open source at any moment, without compromising any credentials.

Note that this definition of “config” does **not** include internal application config, such as `config/routes.rb` in Rails, or how code modules are connected in Spring. This type of config does not vary between deploys, and so is best done in the code.

Another approach to config is the use of config files which are not checked into revision control, such as `config/database.yml` in Rails. This is a huge improvement over using constants which are checked into the code repo, but still has weaknesses: it's easy to mistakenly check in a config file to the repo; there is a tendency for config files to be scattered about in different places and different formats, making it hard to see and manage all the config in one place. Further, these formats tend to be language- or framework-specific.

The twelve-factor app stores config in *environment variables* (often shortened to *env vars* or *env*). Env vars are easy to change between deploys without changing any code; unlike config files, there is little chance of them being checked into the code repo accidentally; and unlike custom config files, or other config mechanisms such as Java System Properties, they are a language- and OS-agnostic standard.

Another aspect of config management is grouping. Sometimes apps batch config into named groups (often called “environments”) named after specific deploys, such as the `development`, `test`, and `production` environments in Rails. This method does not scale cleanly: as more deploys of the app are created, new environment names are necessary, such as `staging` or `qa`. As the project grows further, developers may add their own special environments like `joes-staging`, resulting in a combinatorial explosion of config which makes managing deploys of the app very brittle.

In a twelve-factor app, env vars are granular controls, each fully orthogonal to other env vars. They are never grouped together as “environments”, but instead are independently managed for each deploy. This is a model that scales up smoothly as the app naturally expands into more deploys over its lifetime.

- Each of the deployment environments we will use has a method for configuring environment variables.
- CI/CD, GitHub actions, etc. can integrate environment variables with the development process.
- Vaults and secrets managers are a better approach for passwords.

IV. Backing Services

IV. Backing services

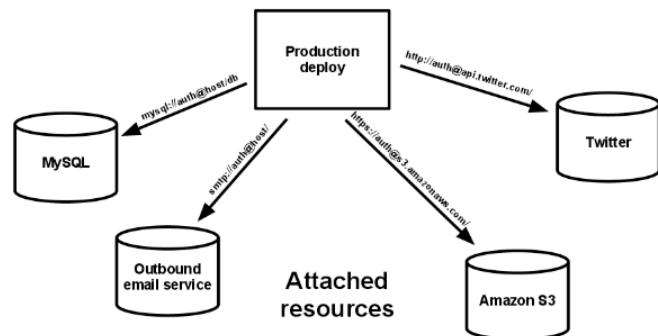
Treat backing services as attached resources

A *backing service* is any service the app consumes over the network as part of its normal operation. Examples include datastores (such as MySQL or CouchDB), messaging/queueing systems (such as RabbitMQ or Beanstalkd), SMTP services for outbound email (such as Postfix), and caching systems (such as Memcached).

Backing services like the database are traditionally managed by the same systems administrators who deploy the app's runtime. In addition to these locally-managed services, the app may also have services provided and managed by third parties. Examples include SMTP services (such as Postmark), metrics-gathering services (such as New Relic or Loggly), binary asset services (such as Amazon S3), and even API-accessible consumer services (such as Twitter, Google Maps, or Last.fm).

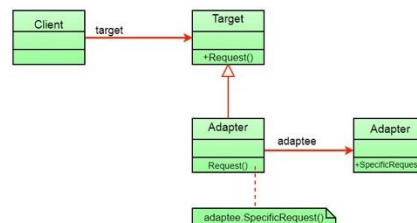
The code for a twelve-factor app makes no distinction between local and third party services. To the app, both are attached resources, accessed via a URL or other locator/credentials stored in the `config`. A deploy of the twelve-factor app should be able to swap out a local MySQL database with one managed by a third party (such as Amazon RDS) without any changes to the app's code. Likewise, a local SMTP server could be swapped with a third-party SMTP service (such as Postmark) without code changes. In both cases, only the resource handle in the config needs to change.

Each distinct backing service is a *resource*. For example, a MySQL database is a resource; two MySQL databases (used for sharding at the application layer) qualify as two distinct resources. The twelve-factor app treats these databases as *attached resources*, which indicates their loose coupling to the deploy they are attached to.



Resources can be attached to and detached from deploys at will. For example, if the app's database is misbehaving due to a hardware issue, the app's administrator might spin up a new database server restored from a recent backup. The current production database could be detached, and the new database attached – all without any code changes.

- My code has shown encapsulating backing services with a shallow adaptor.
- This is an example of the *adaptor pattern*.



- You seen examples of things like `AbstractBaseDataService`,
- The approach is also an application of several aspects of SOLID.
(<https://en.wikipedia.org/wiki/SOLID>)